

Conditional Statements

if statements

An if statement consists of a boolean expression followed by one or more statements.

if...else statements

An if statement can be followed by an optional else statement, which executes when the boolean expression is FALSE.

nested if statements

You can use one if or else if statement inside another if or else if statement(s).

Indentation

- Increase indent after a if statement or for statement.
- Maintain indent to indicate the scope of the block (which lines are affected by the if/for).
- Reduce indent back to the level of the if statement or for statement to indicate the end of the block.
- Blank lines are ignored - they do not affect indentation.
- Comments on a line by themselves are ignored with regard to indentation.

Example-1:

```
x = 5
if x < 10:
    print 'Smaller'
if x > 20:
    print 'Bigger'
```

Example-2:

```
x = 5
if x < 2 :
    print 'small'
elif x < 10 :
    print 'Medium'
```

else :

 print 'LARGE'

print 'All done'

Example 3:

Q) Write pay computation to give the employee 1.5 times the hourly rate for hours worked above 40 hours (Input: hours and rate).

```
rate = input("Rate:")
```

```
hours = input("Hours:")
```

```
if hours <= 40:
```

```
    pay = hours * rate
```

```
else:
```

```
    pay = 40 * rate + 1.5 * rate * (hours - 40)
```

```
print("Pay :", pay)
```

Python Try Except

- The try block lets you test a block of code for errors.
- The except block lets you handle the error.
- The else block lets you execute code when there is no error.
- The finally block lets you execute code, regardless of the result of the try- and except blocks.

Exception Handling

When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

These exceptions can be handled using the try statement:

The try block will generate an exception, because x is not defined:

```
try:
    print(x)
except:
    print("An exception occurred")
```

Since the try block raises an error, the except block will be executed.

Without the try block, the program will crash and raise an error
This statement will raise an error, because x is not defined

Many Exceptions

You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error:

Example

Print one message if the try block raises a NameError and another for other errors:

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
```

Else

You can use the else keyword to define a block of code to be executed if no errors were raised:

In this example, the try block does not generate any error:

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

Finally

The finally block, if specified, will be executed regardless if the try block raises an error or not.

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

Raise an exception

As a Python developer you can choose to throw an exception if a condition occurs.

To throw (or raise) an exception, use the raise keyword.

Example

Raise an error and stop the program if x is lower than 0:

```
x = -1
```

```
if x < 0:
```

```
    raise Exception("Sorry, no numbers below zero")
```

The raise keyword is used to raise an exception.

You can define what kind of error to raise, and the text to print to the user.

Example

Raise a TypeError if x is not an integer:

```
x = "hello"
```

```
if not type(x) is int:
```

```
    raise TypeError("Only integers are allowed")
```

Q) Write a program which takes input of a number and check whether it is even or odd.

```
try:
```

```
    x = input("Enter a number : ")
```

```
except:
```

```
    x = 'n'
```

```
if x is 'n':
```

```
    pass
```

```
elif x%2 == 0:
    print("Even")
else:
    print("Odd")
```

Assignment:

1. Write a program which takes two integers as input. Print whether the two numbers are equal, less than, or greater than the other.
2. Write pay computation to give the employee 1.5 times the hourly rate for hours worked above 40 hours (Input: hours and rate).
3. Write a program to prompt for a score between 0.0 and 1.0. If the score is out of range, print an error message. If the score is between 0.0 and 1.0, print a grade using the following table:
>= 0.9 A
>= 0.8 B
>= 0.7 C
>= 0.6 D
< 0.6 F
4. Write a program which takes temperature as input and also input its type (Centigrade or Fahrenheit). Convert it to other format.
5. Write a program which takes input of a number and check whether it is positive or negative.
6. Write a program which takes input of a number and check whether it is even or odd.
7. Write a program which takes three numbers as input find maximum of three and minimum of the three.
8. Write pay program using try and except so that your program handles non-numeric input gracefully by printing a message and exiting the program. The following shows two executions of the program:
Enter Hours: 20

Enter Rate: nine

Error, please enter numeric input

Enter Hours: forty

Error, please enter numeric input

Questionnaires:

1. Write down the advantages and disadvantages of conditional statements in Python compared to C.

Study on Loop Control Structures

while loop

Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.

a) while expression:

statement(s)

Example:

```
count = 0
while (count < 3):
    count = count + 1
    print("Hello")
```

b) while condition:

execute these statements

else:

execute these statements

Example:

```
count = 0
while (count < 3):
    count = count + 1
    print("Hello")
```

```
else:  
    print("In Else Block")
```

for loop

Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

```
for iterator_var in sequence:  
    statements(s)
```

Example:

```
n = 4  
for i in range(0, n):  
    print(i)
```

```
list = ["apple", "orange", "mango"]  
for index in range(len(list)):  
    print(list[index])
```

nested loops

You can use one or more loop inside any another while, for or do..while loop.

a) for iterator_var in sequence:

for iterator_var in sequence:

statements(s)

statements(s)

b) while expression:

while expression:

statement(s)

statement(s)

Example:

```
for i in range(1, 5):
    for j in range(i):
        print(i, end=' ')
    print()
```

break statement

Terminates the loop statement and transfers execution to the statement immediately following the loop.

```
for letter in 'debashis':

    # break the loop as soon it sees 'e'
    # or 's'
    if letter == 'e' or letter == 's':
        break

print('Current Letter :', letter)
```

continue statement

```
while True:
    ...
    if x == 10:
        continue
    print(x)
```

Example:

```
for var in "debashis":
    if var == "e":
        continue
    print(var)
```

Example:

```
for i in range(1, 11):
    # If i is equals to 6,
    # continue to next iteration
    # without printing
    if i == 6:
        continue
    else:
        # otherwise print the value
        # of i
        print(i, end=" ")
```

Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

pass statement

```
for letter in 'debashis':
    pass
print('Last Letter :', letter)
```

The pass statement in Python is used when a statement is required

syntactically but you do not want any command or code to execute.